

Summary of "Pipeline Parallelism with Controllable Memory"

Letian Ruan(ruanlt), Terry Shi (weiceica), Fu-Shiang Yang(fsyang)

Problem and Motivation

Pipeline parallelism has been widely explored, but most existing schedules lack a systematic methodology. By decomposing pipeline schedules, the authors find all existing methods are inefficient. Therefore, they introduce a family(V-shape) of building blocks to tackle the activation peak memory problem.

Related Works

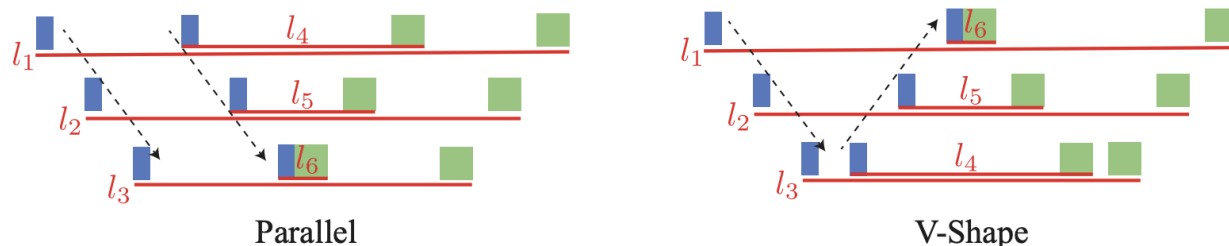
Gpipe - Reducing the bubble rate by increasing the number of microbatches, at the cost of more activation memory

1F1B - Avoiding the activation memory growth with respect to the number of microbatches by staggering forward pass and backward pass, keeping the same bubble rate with GPipe

ZB-2H - Achieving zero bubble rate by splitting the backward into activation gradient computation and weight gradient computation. However, it doubles the peak memory compared to 1F1B

Solution Overview

In the following formulation, the peak memory of each device is decided by the sum of lifespans. peak memory of device $i \leq Xs \in Si[l_i/T_i]$ ms



Basically, because the passed down parallel building blocks cause stage 1 having a higher sum of lifespan(l_1+l_4), they convert the direction to ensure balanced lifespan(l_1+l_6) in every stage. Then, the authors propose V-Min, V-Half, and V-ZB pipeline schedules, which have different offset between forward and backward process. (e.g. V-Min: $\delta_0=1$ & $\delta_1=1$, V-Half: $\delta_0=2$ & $\delta_1=1$, and V-ZB: $\delta_0=4$ & $\delta_1=2$). By doing so, the peak memory and the bubble rate would be V-Min: $M/3$ & $4d$, V-Half: $M/2$ & $3d$, and V-ZB: M & 0 compared to 1F1B(M & $6d$).

Limitations

V-Shape methods require doubled communication cost between pipeline stages. Although the authors claim this cost is relatively small and can be neglected, it may become a bottleneck in bandwidth-limited environments

Future Research Directions

A major limitation of V-Min is that it suffers from growing bubbles when increasing the number of microbatches. Although V-Half mitigates this issue, there is still a space to further reduce the memory consumption. Using continuous offsets or finer-granularity discretization is a possible way to solve it

Summary of Class Discussion

Q: What does m in this equation represent

$$\text{peak memory of device } i \leq \sum_{s \in S_i} \lceil \frac{l^s}{T} \rceil m^s$$

A: M is the size of activation memory for a single microbatch

Q: Can we add checkpoint for the activation to improve V-shape pipeline

A: It may be misnormal for using checkpoint to improve because it is usually used for reducing memory usage and false tolerance. Whether to add depends on the devices users use. It is a tradeoff for better balancing resources.

Q: While the models scale up, communication costs between pipeline stages maybe higher?

A: Yes, the communication might be higher when model parameters scale up. So when deciding which parallel strategy to use, we need to firstly detect the bottleneck. If there is more bandwidth left, we can use communication-bound pipeline parallelism. And if in the future, memory becomes the cheapest, we can also choose other methods that are memory bound.

Q: Would they spend the same time for forward and backward processes in a realistic scenario?

A: Basically, the backward process takes approximately double the time of the forward process, which is why they split the backward process into two parts (B and W). This makes the building blocks(F, B, W) approximately equal in duration.

Q: How to swap the stage with a long lifespan with the short one in V-shape building block?

Does V-shape introduce any new method to break the dependencies?

A: No, V-shape building block still follows the original dependencies. It just changes the placement of model parameters and the execution sequences on each device.

Summary of "WLB-LLM: Workload-Balanced 4D Parallelism for Large Language Model Training"

Letian Ruan(ruanlt), Terry Shi (weiceica), Fu-Shiang Yang(fsyang)

Problem and Motivation

Training large language models (LLMs) like LLaMA-70B requires massive GPU clusters, but efficiency is hampered by workload imbalance. In large-scale jobs such as 8K-GPU LLM training with a context window of 128K, the slowest GPU may take up to $1.44\times$ longer than others, leaving many devices idle. The root cause for the workload imbalance lies in the input-dependent nature of attention computation and highly varying document lengths in the input sequences. These facts introduce heterogeneity in per-token arithmetic intensity, as tokens at the tail positions of long documents require more attention computation.

However, existing LLM training frameworks fail to recognize the above heterogeneity. Specifically, state-of-the-art LLM training solutions employ a 4D parallelism paradigm that combines data parallelism (DP), pipeline parallelism (PP), context parallelism (CP), and tensor parallelism (TP). At the PP level, input documents are packed into fixed-length micro-batches equal to the context window size. Although this guarantees that each PP worker processes the same number of tokens, it fails to account for the quadratic cost of attention computation, causing some pipeline stages to process much heavier workloads and lengthening the critical path. At CP level, sequence sharding distributes chunks unevenly, making shards that contain document tails much heavier. And since CP involves synchronized AllGather/ReduceScatter communication, the slowest worker determines the group's overall latency.

Related Works

Distributed LLM Training Frameworks:

To address the challenges of training extra-large LLMs, prior approaches primarily rely on 3D parallelism (TP+PP+DP). Although 3D parallelism has proven its effectiveness in scaling model size, enabling the training of models with trillions of parameters, it struggles to scale context window size effectively. To address this limitation, a new dimension of parallelism—context parallelism—has been introduced, forming the 4D parallelism training paradigm. Context parallelism can split the input context into chunks, effectively reducing the memory bottleneck associated with very long sequence-length inputs.

Input Padding and Packing for LLM Training:

The input of LLMs consists of samples with varying lengths. To batch input documents together and optimize GPU utilization during LLM training, the input documents must be organized into tensors of identical lengths. This can be achieved through two primary approaches: Padding and Packing. Padding involves adding zero padding to shorter documents within a micro-batch. However, this approach inevitably introduces redundant computation, communication, and memory overhead. To address this issue, prior work has focused on designing more efficient kernels to reduce redundant computation or optimizing batching strategies to minimize padding. To fully eliminate redundant computation, recent works propose packing short documents together to form a single long input sequence. After packing, an additional attention mask must be applied to ensure tokens only attend to others within the same document. Due to its efficiency in avoiding redundant computation, packing has become the mainstream choice in LLM training.

Solution Overview

Pipeline Parallelism Solutions: At the PP level, WLB-LLM relaxes the fixed-length constraint on micro-batches and introduces variable-length packing to balance total workload, considering attention, GEMM, and communication costs together. To handle extremely long documents that disproportionately slow down training, it applies outlier document delay, placing them into waiting queues and later distributing them evenly across micro-batches. Since these long documents make up only a small fraction of tokens, the delay minimally affects randomness while preventing bottlenecks. To make this practical, WLB-LLM integrates a heuristic packing algorithm that combines these strategies efficiently at runtime, producing near-optimal balance with negligible overhead.

Context Parallelism Solutions: At the CP level, the system mitigates shard-level imbalance with per-document sharding, which splits documents symmetrically so each worker receives equal computation. Yet, finer sharding can reduce GPU kernel efficiency due to wasted tile-level operations. To resolve this tension, WLB-LLM employs adaptive sharding selection, dynamically predicting kernel latency and choosing between per-sequence and per-document strategies. This ensures each micro-batch uses the most efficient sharding approach. Together, these PP- and CP-level techniques reduce straggler effects across GPUs and improve end-to-end training throughput.

Experiments show that WLB-LLM delivers an average 1.23× training speedup across models up to 70B parameters, without harming convergence.

Limitations

The limitations of the WLB-LLM system include the following:

1. The current adaptive sharding selection is coarse-grained, as it only picks between per-sequence and per-document sharding at the whole-sequence level. A finer-grained hybrid scheme could dynamically adjust cut positions, applying per-sequence to short segments and per-document to long ones, thereby reducing overhead and improving efficiency.
2. If the input data does not exhibit a highly skewed document length distribution, the new mechanisms in WLB-LLM may provide limited benefit while introducing additional computation overhead and scheduling complexity.
3. For now, WLB-LLM only supports dense models and lacks deeper support for Mixture-of-Experts (MoE) models. Although compatible with expert parallelism, it does not address expert-level load imbalance from uneven token routing. This limitation is significant because MoE models have become the dominant paradigm in large-scale LLM training..

Future Research Directions

- Future work could develop hybrid approaches that dynamically adjust cut positions, applying per-sequence sharding to short segments and per-document sharding to long ones for better efficiency.
- Extend WLB-LLM to support MoE models, where expert parallelism introduces severe load imbalance from uneven token routing. Inspired by prior work such as eplb, future research could integrate workload-aware packing with expert routing or design joint balancing strategies across document- and expert-level workloads.
- Future work should investigate how extremely long documents influence workload distribution, scheduling efficiency, and memory usage.

Summary of Class Discussion

Q: How to implement the outlier delay? Why does this technique make less impact on the model convergence?

A: The system implements a multi-level waiting queue for outlier documents. The execution of documents in outlier queues is delayed until sufficient documents accumulate. Once a queue reaches the number of micro-batches in a global batch (N documents), the outlier documents are released from the queue and distributed to the current batch processing. However, there's no detailed explanation why the technique has less impact on the model convergence.

Q: How does WLB-LLM profile the latency of per-doc and per-seq strategies in adaptive sharding selection?

A: First, they calculate the total floating point operations required for the attention computation. Next, they estimate the achieved TFLOPs for the given tensor shape using the data collected from offline profiling which includes the impact of TMA usage. Finally, the attention kernel latency is estimated by dividing the amount of floating point operations by the achieved TFLOPs.

Q: When failure happens, what is the recovery process for model training?

A: The current approach relies on frequent checkpoints and maintaining detailed state variables, allowing the system to roll back to stable training states when errors are detected.

Q: What is the context parallel? Does it transfer KV cache between different workers?

A: Yes. Context parallelism splits the input sequence across GPUs instead of splitting weights or layers. Each GPU processes only a slice of the tokens, reducing memory usage for long contexts. Since attention requires every token to see all previous ones, KV cache slices must be exchanged between workers using AllReduce or ReduceScatter.

Q: Why doesn't WLB-LLM introduce the pre-pack mechanism to reduce computation? What is the overhead for storing pre-pack data?

A: Directly storing the pre-packed dataset for training would break the randomness of input data. However, using metadata to store pre-solved ILP packing strategies can reduce runtime packing latency while introducing little overhead. This is a promising direction for future work.